Introduction to Java

COMP1039 Programming Paradigms

Yuan Cheng

Spring 2026

School of Computer Science
The University of Nottingham Ningbo China

Introduction

What This Module Is About?

- What is a **programming paradigm**?
- Why are we starting with **Java**?

What Is in the Java Part of This Module?

Textbook Coverage:

- Chapters 1–11 of *Java Programming: A Comprehensive Introduction* Herbert Schildt and Dale Skrien (2013)

Main Topics:

1. Fundamentals
2. Classes, Objects, Methods
3. Inheritance
4. Interfaces
5. Exception Handling and I/O
6. Packages

What to Expect:

- A **very basic introduction** to Java and OOP
- We assume **no prior Java experience**
- Many useful online resources are available

What is Java?

The Java buzzword-bingo!

- Java is a *simple*, *object-oriented*, *distributed*, *interpreted*, *robust*, *secure*, *architecture-neutral*, *portable*, *high-performance*, *multi-threaded*, *dynamic* language.

Right Language, Right Time

- Kept all the good features of C/C++
- Dumped a lot of the ugliness
- Timing:
  - Internet boom
  - Moore's Law

Origins and Strengths of Java

Origins

- Created in 1991 at Sun Microsystems by James Gosling and his team
- Initially called *Oak*; renamed **Java** in 1995
- Designed for a networked, distributed world

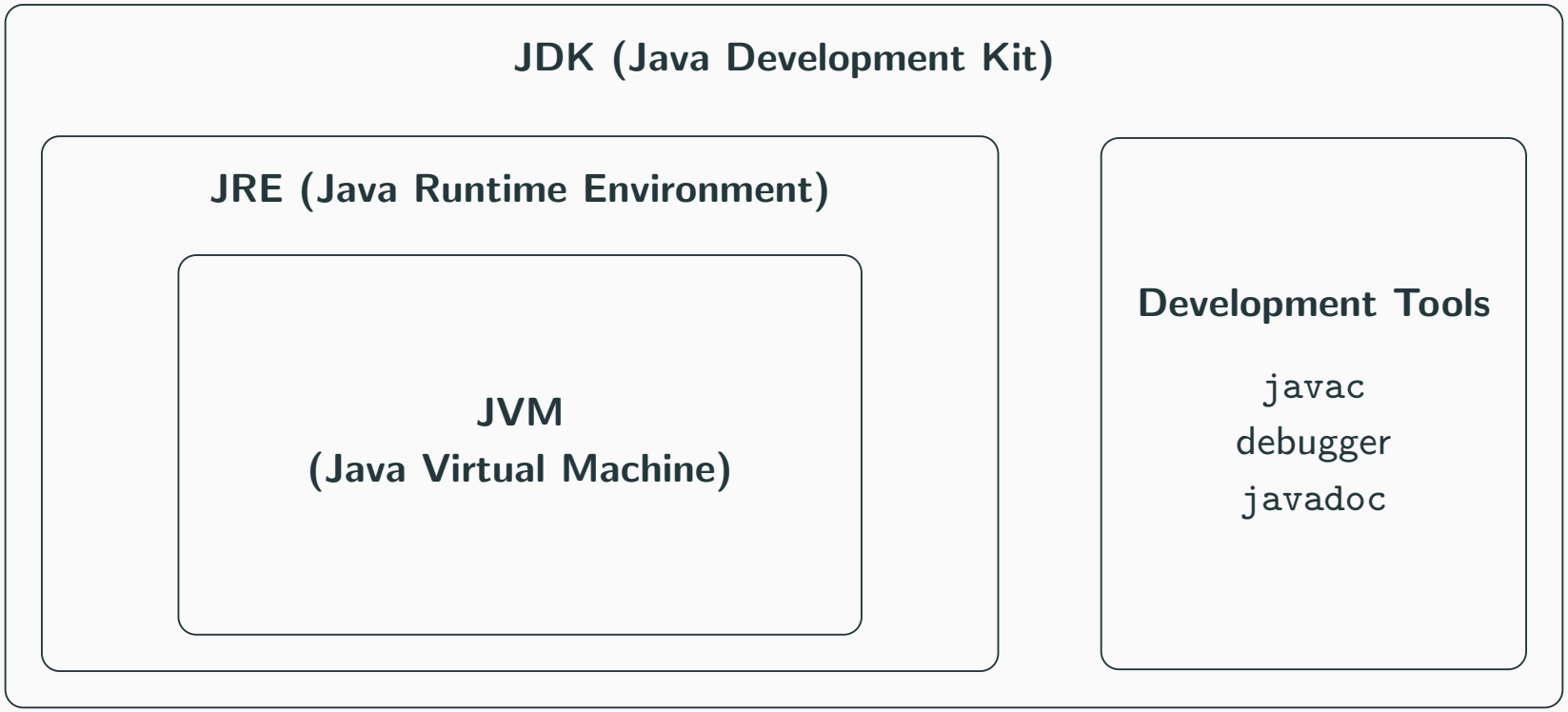
Evolution and Impact

- Quickly adopted for web and enterprise systems
- Widely used today in backend services, big data platforms, and Android ecosystems

Core Strengths

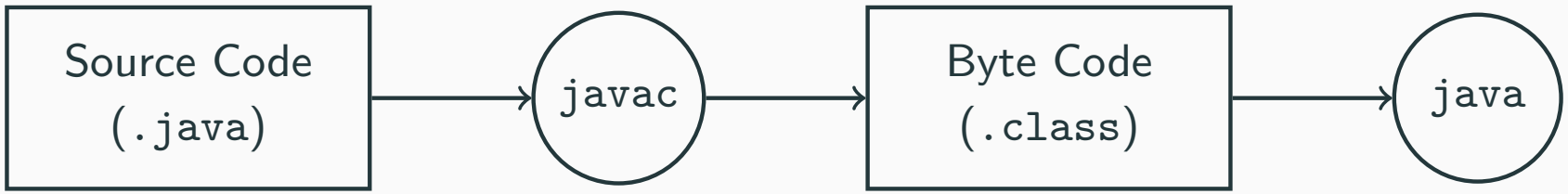
- **Portability** — "Write Once, Run Anywhere"
- **Safety** — strong typing + runtime verification
- **Robustness** — automatic memory management
- **Simplicity** — reduced complexity compared to C++

JDK, JRE, and JVM



- To **run** Java programs: you need the **JRE**
- To **develop** Java programs: you need the **JDK**

Compiling Java



- Write source code in a `.java` file
- Compile with `javac` → bytecode (`.class`)
- Run with `java` (JVM executes the bytecode)

First Program

```
1  /*
2      This is a simple Java program.
3      Call this file HelloWorld.java.
4  */
5  public class HelloWorld {
6      // A Java program begins with a call to main().
7      public static void main(String[] args) {
8          System.out.println("Hello World.");
9      }
10 }
```

Comments

```
1  /*
2      This is a simple Java program.
3      Call this file HelloWorld.java.
4  */
5  public class HelloWorld {
6      // A Java program begins with a call to main().
7      public static void main(String[] args) {
8          System.out.println("Hello World.");
9      }
10 }
```

Multi-line Comment

Single-line Comment

Declaring a Java Class

```
1  class MyClass {
2
3      // members of the class go here
4
5  }
```

- `class`
  - A keyword (reserved word) used to declare a class
- **Class Name**
  - An identifier
  - By convention:
    - Begins with a capital letter
    - Capitalise the first letter of each word (e.g., `HelloWorld`)
- { } **Braces**
  - Define the scope of the class
  - Group related statements together
  - Everything inside the braces is a **member** of the class

The `main` Method

```
1 public static void main(String[] args) {
2     // program starts here
3 }
```

- **Parentheses ()**
  - Indicate this is a programming building block called a **method**
- **Methods**
  - Are defined inside a **class**
  - Contain statements that perform tasks
- `main`
  - The starting point of every Java application
  - When the program starts, the JVM looks for `main`
  - Every standalone Java application must have exactly one `main` method

Breaking Down the `main` Method

```
1 public static void main(String[] args)
```

- `public`
  - Access modifier
  - Allows the method to be called from outside the class
  - The `main` method *must* be declared public
- `static`
  - Belongs to the class, not to an object
  - The JVM calls `main` without creating an object
- `void`
  - The method does not return any value
- `main`
  - The name of the starting method
- `String[] args`
  - Parameter list
  - An array of `String` objects
  - Used to receive command-line arguments

The `println` Statement

```
1 System.out.println("Hello World.");
```

- `System`
  - A pre-defined class in Java
  - Provides access to system-level resources
- `out`
  - An output stream connected to the console
- `println()`
  - A built-in method
  - Displays the given string
  - Prints a newline after the output
- **Semicolon ;**
  - Terminates the statement
  - All Java statements end with a semicolon

Handling Syntax Errors

Types of Errors

- **Syntax Errors**
  - Errors in the grammar (syntax) of the programming language
  - Detected at **compile time**
  - The Java compiler reports these errors before execution
- **Runtime Errors**
  - Errors that occur while the program is running
  - The program compiles successfully but fails during execution
  - (We will study these later)

Common Causes of Syntax Errors

- Spelling mistakes (e.g., `publc` instead of `public`)
- Using reserved keywords as identifiers
- Incorrect characters in identifiers
- Case differences (Java is case-sensitive!)
- Missing symbols such as { or ;

Second Program: Using Variables

```
1  /* This demonstrates a variable.
2     Call this file Example2.java. */
3  class Example2 {
4
5      public static void main(String[] args) {
6          int var1; // this declares a variable
7          int var2; // this declares another variable
8
9          var1 = 1024; // this assigns 1024 to var1
10
11         System.out.println("var1 contains " + var1);
12
13         var2 = var1 / 2;
14
15         System.out.print("var2 contains var1 / 2: ");
16         System.out.println(var2);
17     }
18 }
```

Types

A Data Type

- Defines the kind of values that can be stored and manipulated

Two Categories

- **Primitive (Non-OO) Types** — 8 built-in types:
  - `byte`, `short`, `int`, `long`
  - `float`, `double`
  - `char`, `boolean`
- **OO (Reference) Types**
  - Objects created from classes (e.g., `String`)

Examples:

- `boolean` → `true`, `false`
- `int` → `0`, `-47`
- `double` → `3.14`
- `String` → `"hello"`

Variables

What is a Variable?

- A **named location** in memory
- Stores a value of a specific **type**

Declaration Form

TYPE VAR-NAME;

Example

```
String foo;
```

Assignment

- Use the = operator to give a variable a value
- The value on the right is assigned to the variable on the left
- Can be combined with a variable declaration

Example 1

```
String foo;

foo = "IAP 6.092";
```

Examples 2 & 3

```
double badPi = 3.14;

boolean isJanuary = true;
```

Operators

- Symbols that perform computations on values
- Operate on variables and expressions

Basic Arithmetic and Assignment Operators

- Assignment: =
- Addition: +
- Subtraction: -
- Multiplication: *
- Division: /

Keywords

`abstract`	`assert`	`boolean`	`break`	`byte`	`case`
`catch`	`char`	`class`	`const`	`continue`	`default`
`do`	`double`	`else`	`enum`	`extends`	`final`
`finally`	`float`	`for`	`goto`	`if`	`implements`
`import`	`instanceof`	`int`	`interface`	`long`	`native`
`new`	`package`	`private`	`protected`	`public`	`return`
`short`	`static`	`strictfp`	`super`	`switch`	`synchronized`
`this`	`throw`	`throws`	`transient`	`try`	`void`
`volatile`	`while`				

- Keywords are **reserved words**
- They cannot be used as identifiers (e.g., variable or class names)

Identifiers

- A name given to a method, variable, class, or other user-defined item

Allowed Characters

- Letters: `A--Z`, `a--z`
- Digits: `0--9`
- Symbols: `$`, `_`
- Unicode characters are allowed (e.g., Chinese characters)

Rules

- Cannot be empty (must contain at least one character)
- The first character **cannot be a digit** (e.g., `9sees` ×)
- Java is **case-sensitive** (e.g., `myvar` and `myVar` are different)

Variable Naming Conventions

General Rules

- Do not use Java **keywords** as names
- No whitespace in identifiers
- Use descriptive names (self-documenting code)

Unicode

- Java allows Unicode characters
- But avoid non-ASCII characters in practice

camelCase Style

- Combine multiple words without spaces
- Capitalize each word after the first
- Example: `topLevel`, `studentScore`

Variable Naming Conventions (cont.)

Type	Rules	Example
Class	Start with uppercase letter (PascalCase); typically a noun/noun phrase.	`class Employee { }`
Interface	Start with uppercase letter (PascalCase); often an adjective (capability) or a noun.	`interface Runnable { }`
Method	Start with lowercase letter; verb/verb phrase; camelCase for multiple words.	`void draw() { }`
Variable	Start with lowercase letter; camelCase for multiple words; descriptive name.	`int id;`
Constant	UPPER_CASE_WITH_UNDERSCORES; may contain digits (not first); typically `static final`.	`static final int MIN_AGE = 18;`

Exercise 1: Spot the Identifiers

Task (30 seconds): In the following Java program, **circle all identifiers**. (Identifiers are names of classes, methods, variables, parameters, etc.)

Code

```
1  public class Identifiers {
2
3      public static void main(String[] args) {
4          int a = 20;
5      }
6
7  }
```

Question: What are the identifiers? What are *not* identifiers (keywords / literals / symbols)?

Exercise 1: Spot the Identifiers (Answer)

Identifiers:

- `Identifiers` (class name)
- `main` (method name)
- `String` (predefined class name; still an identifier)
- `args` (parameter name)
- `a` (variable name)

Not identifiers:

- `public`, `class`, `static`, `void`, `int` (keywords)
- 20 (integer literal)
- `{`, `}`, `(`, `)`, `;` (symbols / punctuation)

Exercise 2: Spot the Illegal Identifiers

Task (45 seconds): Which declarations contain **illegal identifiers**? Mark each line as **LEGAL** or **ILLEGAL**.

Code

```
1 int 2value;
2 int student-score;
3 int class;
4 int studentScore;
5 int _count;
6 int $amount;
```

Question: If an identifier is legal, is it necessarily *good style*?

Exercise 2: Spot the Illegal Identifiers (Answer)

ILLEGAL:

- `2value` (cannot start with a digit)
- `student-score` (hyphen - not allowed)
- `class` (reserved keyword)

LEGAL (but style varies):

- `studentScore` (recommended camelCase)
- `_count` (legal; underscore allowed)
- `$amount` (legal; $ allowed but discouraged)

Note: *Legal $\neq$ good style.*

Exercise 3: Case Sensitivity Trap

Task (20 seconds): Are these the **same variable**?

Code

```
1 int count;
2 int Count;
```

Follow-up: Even if it compiles, should you do this in real code?

Exercise 3: Case Sensitivity Trap (Answer)

Answer: No. They are **different** identifiers.

Reason: Java is **case-sensitive**. `count` $\neq$ `Count`.

Best practice: Avoid identifiers that differ only by case.

Exercise 4: Convention Challenge

Task (30 seconds): Which option follows **Java naming conventions**?

Class name

```
1 class employeeRecord {}
2 class EmployeeRecord {}
```

Variable name

```
1 int student_score;
2 int studentScore;
```

Question: State the rule you are using (PascalCase vs camelCase).

Exercise 4: Convention Challenge (Answer)

Recommended answers:

- Class name: `EmployeeRecord` (PascalCase)
- Variable name: `studentScore` (camelCase)

Rule of thumb:

- **Classes / Interfaces:** PascalCase
- **Methods / Variables:** camelCase
- **Constants:** UPPER_CASE_WITH_UNDERSCORES

Java Class Libraries

Built-in Methods and Classes

- Methods like `print()` and `println()` belong to the `System` class
- `System` is a predefined class automatically available in Java

The Java Standard Library

- Java includes extensive built-in class libraries
- Provide functionality for:
  - Input/Output (I/O)
  - String handling
  - Networking
  - Graphical User Interfaces (GUI)

Key Idea:

- Java = **Language** + **Standard Class Libraries**
- Becoming a Java programmer means learning to use these libraries

Exit Slips

- Take **1–2 minutes** to reflect on this lecture.
- On a sheet of paper, write:
  - **One thing you learned** in this lecture
  - **One thing you didn't understand**